

Lecture 11: Off-Policy and Multi-Step Learning

Hado van Hasselt
UCL

2021

Background reading: Sutton & Barto 2018, Chapters 5, 7, 11.

1 Overview

Previous lectures:

- Model-free prediction & control.
- Multi-step updates (and eligibility traces).
- Understanding dynamic programming operators.
- Predictions with function approximation.
- Model-based algorithms.
- Policy gradients and actor-critic algorithms.

This lecture:

- Off-policy learning, especially when combined with multi-step updates and function approximation.

2 Motivation for Off-Policy Learning

Off-policy learning is important to learn about hypothetical, counterfactual events (i.e., “what if” questions). Use cases include:

- Learning about the greedy policy (e.g., Q-learning).
- Learning about (many) other policies simultaneously.
- Learning from observed data (e.g., stored logs or other agents).
- Learning from past policies.

Off-policy corrections are also important to correct for mismatch in data distributions, for instance when using policy gradients.

2.1 One-Step Off-Policy Learning

With action values, one-step off-policy learning is relatively straightforward. The general update rule is:

$$q(S_t, A_t) \leftarrow q(S_t, A_t) + \alpha_t \left(R_{t+1} + \gamma \sum_a \pi(a | S_{t+1}) q(S_{t+1}, a) - q(S_t, A_t) \right). \quad (1)$$

Example 2.1 (Instances of One-Step Off-Policy). Several well-known algorithms are special cases of (1):

- **Q-learning:** let π be the greedy policy, so that $\sum_a \pi(a | s) q(s, a) = \max_a q(s, a)$.
- **Expected Sarsa:** let π be the current behaviour policy.
- **Sarsa:** let π put all probability mass on the action the behaviour policy actually selected.

2.2 Multi-Step Off-Policy with Importance Sampling

For multi-step updates, we can use importance-sampling corrections. For example, for a Monte Carlo return on a trajectory $\tau_t = \{S_t, A_t, R_{t+1}, \dots, S_T\}$, define:

$$\hat{G}_t \equiv \frac{p(\tau_t | \pi)}{p(\tau_t | \mu)} G_t = \frac{\pi(A_t | S_t)}{\mu(A_t | S_t)} \dots \frac{\pi(A_T | S_T)}{\mu(A_T | S_T)} G_t. \quad (2)$$

Proposition 2.2 (Unbiasedness of Importance-Sampled Return). *The importance-sampled return defined in (2) satisfies*

$$\mathbb{E}[\hat{G}_t | \mu] = \mathbb{E}[G_t | \pi].$$

Remark 2.3 (Multi-Step Efficiency). Multi-step updates often propagate information more efficiently than one-step bootstrapping. Neither full Monte Carlo nor one-step bootstrapping is typically the best trade-off.

2.3 Off-Policy Corrections for Policy Gradients

For policy gradient methods we want to sample or estimate

$$\mathbb{E}[q_\pi(S_t, A_t) \nabla \log \pi(A_t | S_t)]. \quad (3)$$

On-policy, we can sample multi-step returns G_t such that $\mathbb{E}[G_t | \pi] \approx q_\pi(s, a)$. However, if the behaviour policy is $\mu \neq \pi$, we might not be following a gradient direction, and off-policy corrections become necessary.

3 Issues in Off-Policy Learning

The following issues especially arise when learning off-policy:

- **High variance** (especially when using multi-step updates).
- **Divergent and inefficient learning** (especially when using one-step updates with function approximation — the deadly triad).

4 Variance of Importance Sampling

A major issue with importance-sampling corrections is high variance.

Proposition 4.1 (Importance-Sampling Expectation). *For a given state s , the one-step importance-weighted reward is unbiased:*

$$\begin{aligned}\mathbb{E}\left[\frac{\pi(A_t | s)}{\mu(A_t | s)} R_{t+1} \mid A_t \sim \mu\right] &= \sum_a \mu(a | s) \frac{\pi(a | s)}{\mu(a | s)} r(s, a) \\ &= \sum_a \pi(a | s) r(s, a) \\ &= \mathbb{E}[R_{t+1} | A_t \sim \pi].\end{aligned}\tag{4}$$

However, the variance will typically be larger than the on-policy estimator, sometimes greatly so.

Remark 4.2 (Infinite Variance). In some cases the variance of an importance-weighted return can be infinite (see Sutton & Barto, Example 5.5).

5 Mitigating Variance

There are multiple ways to reduce variance. We discuss three techniques:

1. Per-decision importance weighting (Section 5.1).
2. Control variates (Section 5.2).
3. Adaptive bootstrapping (Section 5.3).

5.1 Per-Decision Importance Weighting

Lemma 5.1 (Uncorrelated Variables Need No Correction). *Consider some state s . For any random variable X that does not correlate with the (random) action A , we have*

$$\mathbb{E}[X | \pi] = \mathbb{E}\left[\frac{\pi(A | s)}{\mu(A | s)} X \mid \mu\right] = \mathbb{E}[X | \mu].$$

Intuitively, if the expectation does not depend on the policy, we do not need to correct.

Proof of Lemma 5.1. Since X and $\frac{\pi(A|s)}{\mu(A|s)}$ are uncorrelated,

$$\begin{aligned}\mathbb{E}\left[\frac{\pi(A | s)}{\mu(A | s)} X \mid \mu\right] &= \mathbb{E}[X | \mu] \mathbb{E}\left[\frac{\pi(A | s)}{\mu(A | s)} \mid \mu\right] \\ &= \mathbb{E}[X | \mu] \sum_a \mu(a | s) \frac{\pi(a | s)}{\mu(a | s)} \\ &= \mathbb{E}[X | \mu] \sum_a \pi(a | s) \\ &= \mathbb{E}[X | \mu].\end{aligned}$$

The last step uses $\sum_a \pi(a | s) = 1$. Similarly, in general, $\mathbb{E}\left[\frac{\pi(A|s)}{\mu(A|s)} \mid \mu\right] = 1$. □

Definition 5.2 (Importance-Sampling Ratio). Define the shorthand notations:

$$\rho_t \equiv \frac{\pi(A_t | S_t)}{\mu(A_t | S_t)}, \quad \rho_{t:t+n} \equiv \prod_{k=t}^{t+n} \rho_k = \prod_{k=t}^{t+n} \frac{\pi(A_k | S_k)}{\mu(A_k | S_k)}. \quad (5)$$

Using Definition 5.2, the reweighted Monte Carlo return from state S_t terminating at time T can be written as

$$\rho_{t:T-1} G_t = \left(\prod_{k=t}^{T-1} \frac{\pi(A_k | S_k)}{\mu(A_k | S_k)} \right) \left(\sum_{k=t}^{T-1} \gamma^{k-t} R_{k+1} \right) = \sum_{k=t}^{T-1} \rho_{t:T-1} \gamma^{k-t} R_{k+1}. \quad (6)$$

We can interpret the importance weight $\rho_{t:T-1}$ as applying to each reward.

Proposition 5.3 (Per-Decision Importance Weighting). *Since earlier rewards cannot depend on later actions,*

$$\mathbb{E}[\rho_{t:T-1} G_t | \mu] = \mathbb{E} \left[\sum_{k=t}^{T-1} \rho_{t:T-1} \gamma^{k-t} R_{k+1} \mid \mu \right] = \mathbb{E} \left[\sum_{k=t}^{T-1} \rho_{t:k} \gamma^{k-t} R_{k+1} \mid \mu \right]. \quad (7)$$

This follows from Lemma 5.1: for each reward R_{k+1} , the ratios $\rho_{k+1}, \dots, \rho_{T-1}$ are uncorrelated with R_{k+1} and can be dropped without changing the expectation.

Definition 5.4 (Per-Decision Importance-Weighted Return). The per-decision importance-weighted return satisfies the recursion:

$$G_t^\rho = \rho_t (R_{t+1} + \gamma G_{t+1}^\rho). \quad (8)$$

This can be used to learn v_π from data generated under $\mu \neq \pi$.

To learn **action values** q_π , we can instead use

$$G_t^\rho = R_{t+1} + \gamma \rho_{t+1} G_{t+1}^\rho. \quad (9)$$

The difference is that when learning $q_\pi(s, a)$, the first action A_t is already conditioned upon, so ρ_t is not needed for the immediate reward R_{t+1} .

5.2 Control Variates

The idea of control variates can be extended to multi-step returns.

Proposition 5.5 (TD Error Decomposition of λ -Return). *Recall the λ -return error (see also earlier lectures on eligibility traces):*

$$\begin{aligned} \delta_t^\lambda &\equiv G_t^\lambda - v(S_t) \\ &= R_{t+1} + \gamma((1-\lambda)v(S_{t+1}) + \lambda G_{t+1}^\lambda) - v(S_t) \\ &= \underbrace{R_{t+1} + \gamma v(S_{t+1}) - v(S_t)}_{=\delta_t} + \underbrace{\gamma\lambda (G_{t+1}^\lambda - v(S_{t+1}))}_{=\delta_{t+1}^\lambda} \\ &= \delta_t + \gamma\lambda \delta_{t+1}^\lambda. \end{aligned} \quad (10)$$

Adding per-decision importance weights to the recursion in Proposition 5.5 yields:

$$\delta_t^{\rho\lambda} = \rho_t (\delta_t + \gamma\lambda \delta_{t+1}^{\rho\lambda}). \quad (11)$$

By design, this includes the $(1-\rho_t)v(S_t)$ control variate terms. This formulation is sometimes called *error weighting* (to contrast with *reward weighting*).

Proposition 5.6 (Control Variate Equivalence). *One can show that*

$$\mathbb{E}[\delta_t^{\rho\lambda} \mid \mu] = \mathbb{E}[G_t^{\rho\lambda} - v(S_t) \mid \mu],$$

where

$$G_t^{\rho\lambda} = \rho_t \left(R_{t+1} + \gamma \left((1 - \lambda) v(S_{t+1}) + \lambda G_{t+1}^{\rho\lambda} \right) \right) \quad (12)$$

is the per-decision importance-weighted λ -return (cf. Definition 5.4).

However, $\delta_t^{\rho\lambda}$ can have lower variance than $G_t^{\rho\lambda} - v(S_t)$.

5.3 Adaptive Bootstrapping

5.3.1 Bootstrapping and the Deadly Triad

Remark 5.7 (Bootstrapping Removes Importance Weights). To learn action values, the per-decision importance-weighted λ -return (cf. (12)) is

$$G_t^{\rho\lambda} = R_{t+1} + \gamma \left((1 - \lambda) \sum_a \pi(a \mid S_{t+1}) q(S_{t+1}, a) + \rho_{t+1} \lambda G_{t+1}^{\rho\lambda} \right).$$

Setting $\lambda = 0$ gives

$$G_t = R_{t+1} + \gamma \sum_a \pi(a \mid S_{t+1}) q(S_{t+1}, a),$$

which contains no importance weights and hence has low variance. However, bootstrapping too much may open us to the *deadly triad*.

Definition 5.8 (Deadly Triad). The *deadly triad* refers to the possibility of divergence when combining:

1. Bootstrapping,
2. Function approximation,
3. Off-policy learning.

Example 5.9 (Divergence Under the Deadly Triad). Consider two states with features $x_1 = 1$ and $x_2 = 2$, so $v(s_1) = w$ and $v(s_2) = 2w$. Suppose there is a single deterministic transition $s_1 \rightarrow s_2$ with reward $r = 0$. If we apply semi-gradient TD only on this transition, the weight update is

$$\begin{aligned} w_{t+1} &= w_t + \alpha_t (r + \gamma v(s_2) - v(s_1)) \nabla v(s_1) \\ &= w_t + \alpha_t (2\gamma - 1) w_t. \end{aligned}$$

When $\gamma > \frac{1}{2}$, the multiplier $(2\gamma - 1) > 0$, so w_t diverges to $+\infty$ or $-\infty$ depending on its initial sign. This illustrates the deadly triad (Definition 5.8).

Example 5.10 (Multi-Step Returns Mitigate Divergence). Extending Example 5.9 with an additional terminal state (so $s_1 \rightarrow s_2 \rightarrow \text{terminal}$ with $r = 0$ everywhere and $v(\text{terminal}) = 0$), using λ -returns gives:

$$\begin{aligned} \Delta w &= \alpha \left(r + \gamma G_t^\lambda - v(s_1) \right) \\ &= \alpha (2\gamma(1 - \lambda) - 1) w. \end{aligned}$$

The multiplier is negative (i.e., the update is stabilising) when $2\gamma(1 - \lambda) < 1$, i.e.,

$$\lambda > 1 - \frac{1}{2\gamma}.$$

For example, when $\gamma = 0.9$, we need $\lambda > 4/9 \approx 0.45$. This shows that if we do not bootstrap too much, learning can be stable.

5.3.2 Adaptive Bootstrapping: ABTD and V-trace

Remark 5.11 (Bootstrapping Trade-Off). Bootstrapping too much risks the deadly triad (Definition 5.8); bootstrapping too little results in high variance from importance-sampling corrections (Section 4). The idea is to bootstrap *adaptively*, only in as much as the data is off-policy.

Starting from the error-weighted return in (11), introduce a time-dependent bootstrap parameter λ_t :

$$\delta_t^{\rho\lambda} = \lambda_t \rho_t (\delta_t + \gamma \delta_{t+1}^{\rho\lambda}). \quad (13)$$

Setting $\lambda_t = 1$ recovers the previous version (11). We can pick λ_t separately at each time step.

Definition 5.12 (ABTD / V-trace Truncation). Choose λ_t such that $\lambda_t \rho_t \leq 1$:

$$\lambda_t = \min\left(1, \frac{1}{\rho_t}\right). \quad (14)$$

The intuition is: when the data is too off-policy (ρ_t is far from one), truncate the sum of errors, which is equivalent to bootstrapping at that time step.

This is known as **ABTD** (Mahmood et al., 2017) or **V-trace** (Espeholt et al., 2018).

Remark 5.13 (Properties of ABTD / V-trace). • In the tabular case, all such methods update towards some mixture of multi-step returns and therefore converge.

- In deep RL, adaptive bootstrapping greatly helps, especially for policy gradients (which do not tolerate biased return estimates well).
- V-trace is widely used in modern distributed RL systems.

5.3.3 Tree Backup

Definition 5.14 (Tree-Backup Return). The tree-backup algorithm provides another way to adaptively bootstrap without importance weights. Starting from the Bellman equation for action values,

$$q_\pi(s, a) = \mathbb{E} \left[R_{t+1} + \gamma \sum_{a'} \pi(a' | S_{t+1}) q_\pi(S_{t+1}, a') \mid A_t = a, S_t = s \right],$$

note that the expectation does not depend on the behaviour policy μ because we condition on a . The tree-backup return samples this and replaces only the selected action's value:

$$G_t = R_{t+1} + \gamma \sum_{a \neq A_{t+1}} \pi(a | S_{t+1}) q(S_{t+1}, a) + \gamma \pi(A_{t+1} | S_{t+1}) G_{t+1}. \quad (15)$$

Here, $\pi(A_{t+1} | S_{t+1})$ plays a role similar to λ in controlling bootstrapping.

Remark 5.15 (Tree-Backup Properties). The tree-backup return (15) is unbiased and has low variance, since no importance-sampling ratios appear. However, it may bootstrap too early (when $\pi(A_{t+1} | S_{t+1})$ is small), so it is not immune to dangers of the deadly triad (Definition 5.8).